Vibenotes on Mathematic Methods for Data Science

somehow by

Andres Lopez

Institute of Investigations on Applied Mathematics and Systems (IIMAS)

October 11, 2025

This page was unintentionally left blank, I never attend to this class.

Contents

1 Linear Algebra and Algorithms **4**

1.1 Numerical Solution of Linear Systems of Equations 4

1.1.1 Iterative Methods: Jacobi and Gauss-Seidel 4

1.1.2 Architectural Analysis: Computational Cost and Parallelism 5

1.1.3 Heuristics and Advanced Intuition 6

1.1.4 Numerical Diagnosis: Implementation and Error Analysis 7

1.2 Direct Methods: LU Factorization 10

1.2.1 Canonical Factorization Variants: Doolittle, Crout, and Cholesky . 11

1.2.2 Implementation and Numerical Stability: Pivoting and Condition Number 13

1.2.3 Application in Data Science: Inversion, Least Squares, and Statistics 14

1.2.4 Numerical Implementation in Python 16

1.2.5 ⋆ Further Variants and Generalizations of LU Factorization 20

1.3 The QR Factorization: The Stable Path to Least Squares 23

1.3.1 Solving Least Squares with Reduced QR 23

1.3.2 Methods of Construction 24

1.3.3 Numerical Implementation in Python 25

Chapter 1

Linear Algebra and Algorithms

1.1 Numerical Solution of Linear Systems of Equations

The problem of solving the linear system $\mathbf{Ax} = \mathbf{b}$ for a given matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ and vector $\mathbf{b} \in \mathbb{R}^n$ is the ultimate foundational problem in computational mathematics. In the realm of large-scale data science, and particularly for systems arising from the discretization of partial differential equations (PDEs) or high-dimensional optimisation, the direct methods that yield solutions in a finite number of arithmetic operations are supplanted by the elegance of Iterative Methods. These methods trade finite termination for asymptotic convergence, leveraging the sparsity and spectral properties inherent in many real-world matrices.

1.1.1 Iterative Methods: Jacobi and Gauss-Seidel

Iterative methods are fundamentally rooted in the theory of fixed-point iteration on a Banach space. We seek a solution $\mathbf{x} \in \mathbb{R}^n$ as the unique fixed point of a contracted mapping.

Definition 1.1.1 (Matrix Splitting and First-Order Scheme)**.** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be an invertible matrix. An iterative method for solving $\mathbf{Ax} = \mathbf{b}$ is generated by a splitting of $\mathbf{A}$ into two matrices,

$$\mathbf{A} = \mathbf{M} - \mathbf{N}$$

where $\mathbf{M}$ is a non-singular matrix that is computationally trivial to invert. The recurrence relation is:

$$\mathbf{x}^{(k+1)} = \mathbf{M}^{-1}\mathbf{N}\mathbf{x}^{(k)} + \mathbf{M}^{-1}\mathbf{b} = \mathbf{T}\mathbf{x}^{(k)} + \mathbf{c}$$

The matrix $\mathbf{T} = \mathbf{M}^{-1}\mathbf{N}$ is the iteration matrix.

For $\mathbf{A}$, we adopt the canonical additive decomposition: $\mathbf{A} = \mathbf{D} - \mathbf{L} - \mathbf{U}$.

The Jacobi Iteration

Definition 1.1.2 (Jacobi Splitting)**.**

$$\mathbf{M}_J = \mathbf{D}, \quad \mathbf{N}_J = \mathbf{L} + \mathbf{U}$$

The iteration matrix is $\mathbf{T}_J = \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})$.

The Gauss-Seidel Iteration

Definition 1.1.3 (Gauss-Seidel Splitting)**.**

$$\mathbf{M}_{GS} = \mathbf{D} - \mathbf{L}, \quad \mathbf{N}_{GS} = \mathbf{U}$$

The iteration matrix is $\mathbf{T}_{GS} = (\mathbf{D} - \mathbf{L})^{-1}\mathbf{U}$.

The Rigor of Convergence: The Spectral Radius Criterion

Theorem 1.1.4 (Fundamental Criterion for Iterative Convergence)**.** *The iterative method converges for every initial vector $\mathbf{x}^{(0)}$ if and only if the spectral radius of the iteration matrix $\mathbf{T}$ satisfies:*

$$\rho(\mathbf{T}) = \max_{\lambda \in \sigma(\mathbf{T})} |\lambda| < 1$$

The asymptotic rate of convergence is $\mathcal{R}(\mathbf{T}) = -\log \rho(\mathbf{T})$.

Theorem 1.1.5 (Convergence via SDD)**.** *If $\mathbf{A}$ is strictly diagonally dominant (SDD), then both $\mathbf{T}_J$ and $\mathbf{T}_{GS}$ converge.*

Lemma 1.1.6 (Spectral Radius Bound by Induced Norm)**.** *This is trivial but needed, let $\mathbf{T} \in \mathbb{C}^{n \times n}$ be an arbitrary square matrix. For every induced matrix norm, denoted $\|\cdot\|$, the spectral radius of $\mathbf{T}$ is bounded above by the value of that norm:*

$$\rho(\mathbf{T}) \leq \|\mathbf{T}\|$$

Remark 1.1.7. Strict Diagonal Dominance of $\mathbf{A}$ is a rigorous sufficient condition for the iterative stability of the matrix splitting. It translates mathematically into the statement that the ℓ_1-norm of the strictly off-diagonal part of $\mathbf{A}$, when normalized by the diagonal entry, is globally bounded by a factor strictly less than unity. Specifically, defining $\mathbf{T}_J$ such that $\mathbf{A} = \mathbf{D}(\mathbf{I} - \mathbf{T}_J)$, the SDD condition guarantees that the induced ℓ_∞ matrix norm of the Jacobi iteration matrix is a proper contraction:

$$\|\mathbf{T}_J\|_\infty = \max_i \sum_{j \neq i} \left| \frac{a_{ij}}{a_{ii}} \right| < 1$$

Since $\rho(\mathbf{T}_J) \leq \|\mathbf{T}_J\|_\infty$, this immediately implies $\rho(\mathbf{T}_J) < 1$. For Gauss-Seidel, the condition is subtler, relating to a non-induced norm bound on the sequential error ratio (the factor C_i), which is also forced below unity by SDD.

Proof Sketch for Gauss-Seidel. By defining the contraction factor C_{i^*} in the ℓ_∞ norm (as derived previously), we show that $\rho(\mathbf{T}_{GS}) \leq \|\mathbf{T}_{GS}\|_\infty \leq \max_i C_i < 1$, with:

$$C_i = \frac{\sum_{j>i} |a_{ij}|}{|a_{ii}| - \sum_{j<i} |a_{ij}|}$$

The condition SDD ensures $C_i < 1$. □

1.1.2 Architectural Analysis: Computational Cost and Parallelism

Beyond mere existence of convergence, the utility of an iterative method in a modern computing environment is governed by its computational complexity and data dependency.

Computational Complexity (Cost per Iteration)

The expense of a single iteration is dominated by the matrix-vector multiplication associated with $\mathbf{N}\mathbf{x}^{(k)}$ and the inversion (application of the inverse) of $\mathbf{M}$.

Proposition 1.1.8 (Cost per Iteration)**.** *For a dense matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, the complexity of a single iteration for both Jacobi and Gauss-Seidel is $\mathcal{O}(n^2)$.*

Proof. Both methods primarily involve computing the product $\mathbf{N}\mathbf{x}^{(k)}$. Since $\mathbf{N}$ is full (of size $n \times n$), this requires n^2 multiplications and $n^2 - n$ additions, leading to a complexity of $\mathcal{O}(n^2)$. The inversion of $\mathbf{M}$ (diagonal for Jacobi, triangular for Gauss-Seidel) is only $\mathcal{O}(n)$ via vector scaling or forward substitution, respectively.

However, in Data Science and Numerical PDE's, $\mathbf{A}$ is often sparse. If $\mathbf{A}$ has $\mathfrak{n}_z$ nonzero entries (where $\mathfrak{n}_z \ll n^2$), the cost of computing $\mathbf{N}\mathbf{x}^{(k)}$ is reduced to $\mathcal{O}(\mathfrak{n}_z)$, which is often $\mathcal{O}(n)$ for structured matrices (e.g., tridiagonal matrices from 1D discretizations, where $\mathfrak{n}_z \approx 3n$). □

Data Dependency and Parallelization

The true dichotomy between Jacobi and Gauss-Seidel emerges in their temporal data dependency, which dictates their amenability to parallel computing.

1. **Jacobi Method (Concurrent):** The i-th component of the new iterate, $x_i^{(k+1)}$, is computed using only the components from the previous iteration, $\mathbf{x}^{(k)}$.

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j\neq i} a_{ij}x_j^{(k)}\right)$$

   Crucially, all $x_i^{(k+1)}$ can be computed simultaneously and independently. This makes the Jacobi method trivially parallelizable, an immense advantage in modern GPU/-multicore architectures.

2. **Gauss-Seidel Method (Sequential):** The i-th component of the new iterate, $x_i^{(k+1)}$, utilizes a mixture: the newly computed components $x_j^{(k+1)}$ for $j < i$ and the old components $x_j^{(k)}$ for $j > i$.

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)}\right)$$

   This explicit dependency of $x_i^{(k+1)}$ on $x_1^{(k+1)}, \ldots, x_{i-1}^{(k+1)}$ creates a sequential bottleneck (a triangular solve), making the standard Gauss-Seidel iteration difficult to parallelize for general matrices $\mathbf{A}$. Only specialized reordering (e.g., Red-Black ordering for grid problems) can restore parallelism.

1.1.3 Heuristics and Advanced Intuition

The Causal Advantage of Gauss-Seidel

The reason $\rho(\mathbf{T}_{GS}) \leq \rho(\mathbf{T}_J)$ often holds (and thus, Gauss-Seidel converges faster) is rooted in its immediate information propagation. The change made in the i-th component is instantly used to refine the $(i+1)$-th component within the same time step $k \to k+1$. Jacobi delays this information until the next time step, $k+1 \to k+2$.

Symmetric Positive Definite Matrices

Definition 1.1.9 (Symmetric Positive Definite (SPD))**.** A matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is Symmetric Positive Definite if $\mathbf{A} = \mathbf{A}^T$ and for all non-zero vectors $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{x}^T\mathbf{A}\mathbf{x} > 0$.

Theorem 1.1.10 (Convergence for SPD Matrices)**.** *If* $\mathbf{A}$ *is Symmetric Positive Definite, the Gauss-Seidel iteration converges. (Note: Jacobi convergence requires an additional condition, such as* $\mathbf{D} - (\mathbf{L} + \mathbf{U})$ *being positive definite, which is often harder to verify than SDD.)*

Remark 1.1.11 (Minimisation View)*.* For SPD matrices, solving $\mathbf{A}\mathbf{x} = \mathbf{b}$ is equivalent to finding the unique minimiser of the quadratic function:

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x} - \mathbf{b}^T\mathbf{x}$$

The Gauss-Seidel method, in this context, is equivalent to Coordinate Descent, where x_i is updated to minimise $f(\mathbf{x})$ while all other components $\mathbf{x}_j$ $(j \neq i)$ are held fixed. Since $f(\mathbf{x})$ is strictly convex, this sequence of exact line searches is guaranteed to converge to the unique global minimiser $\mathbf{x}$.

The Role of Ordering and Scaling

- **Reordering (Permutation):** The iteration matrices $\mathbf{T}_{GS}$ and $\mathbf{T}_J$ depend explicitly on the matrix $\mathbf{A}$. If we permute the rows and columns via a permutation matrix $\mathbf{P}$ such that the system becomes $(\mathbf{P}\mathbf{A}\mathbf{P}^T)(\mathbf{P}\mathbf{x}) = \mathbf{P}\mathbf{b}$, the new matrix $\mathbf{A}' = \mathbf{P}\mathbf{A}\mathbf{P}^T$ can have drastically different convergence properties. Reordering to maximize the concentration of large coefficients on the diagonal (to approach SDD) can be crucial.

- **Preconditioning:** The core idea of Preconditioning is to replace $\mathbf{A}\mathbf{x} = \mathbf{b}$ with an equivalent system $\mathbf{P}^{-1}\mathbf{A}\mathbf{x} = \mathbf{P}^{-1}\mathbf{b}$ where $\mathbf{P}^{-1}\mathbf{A}$ has a much smaller condition number $\kappa(\mathbf{P}^{-1}\mathbf{A})$ and, critically, an iteration matrix $\mathbf{T}'$ with a smaller spectral radius $\rho(\mathbf{T}')$. The iterative methods discussed here are the simplest forms of preconditioners for themselves (e.g., Jacobi is the Diagonal Preconditioner).

1.1.4 Numerical Diagnosis: Implementation and Error Analysis

The final step is the rigorous implementation, which requires a precise definition of stopping criteria to ensure the computed solution $\mathbf{x}^{(k)}$ is an acceptable approximation of the true solution $\mathbf{x}$.

Numerical Implementation in Python

The true essence of iterative methods manifests in their computational implementation. We employ the `numpy` library for vectorized operations, acknowledging the fundamental trade-off: **Jacobi** benefits immensely from vectorization (parallelism), while **Gauss-Seidel** fundamentally requires sequential processing (causality).

Jacobi Iteration: Vectorized (Trivially Parallel) The Jacobi method is ideally suited for vectorization, as the calculation of $\mathbf{x}^{(k+1)}$ depends only on $\mathbf{x}^{(k)}$, allowing simultaneous computation of all components. This formulation directly expresses the matrix splitting $\mathbf{D}\mathbf{x}^{(k+1)} = (\mathbf{L}+\mathbf{U})\mathbf{x}^{(k)} + \mathbf{b}$.

```
import numpy as np

def jacobi_solve(A, b, x0, max_iter, tol=1e-6):
    """
    Solves Ax = b using the Jacobi method. M = D, N = L + U.
    Implementation is highly vectorized (O(n^2) or O(n) for sparse A).
    """
    n = A.shape[0]
    x_old = np.array(x0, dtype=A.dtype) # Initialize iterate

    # Extract D and R = L + U (R is off-diagonal)
    D = np.diag(A)
    R = A - np.diag(D)

    for k in range(1, max_iter + 1):
        # x_new = D^{-1} * (b - R * x_old)
        # @ is matrix multiplication; / D is component-wise division (D^{-1} operati
        x_new = (b - R @ x_old) / D

        # Relative Error Criterion (L_inf norm)
        error_norm = np.linalg.norm(x_new - x_old, np.inf)
        if error_norm < tol * np.linalg.norm(x_new, np.inf):
            # print(f"Jacobi converged in {k} iterations.")
            return x_new, k

        x_old = x_new

    # print(f"Jacobi failed to converge in {max_iter} iterations.")
    return x_old, max_iter
```

Gauss-Seidel Iteration: Explicit Sequential Loop (Causality) The Gauss-Seidel method **cannot** be fully vectorized in its canonical form because each $x_i^{(k+1)}$ depends causally on $x_j^{(k+1)}$ for $j < i$. This necessitates the use of a sequential loop over the components $i = 1, \ldots, n$, explicitly revealing the data dependency constraint (the lower-triangular solve).

```
def gauss_seidel_solve(A, b, x0, max_iter, tol=1e-6):
    """
    Solves Ax = b using the Gauss-Seidel method. M = D - L, N = U.
    Requires an explicit sequential loop (forward substitution).
    """
    n = A.shape[0]
    x_old = np.array(x0, dtype=A.dtype)
```

```
    for k in range(1, max_iter + 1):
        x_new = np.copy(x_old)

        # Sequential update: The causal heart of the method
        for i in range(n):
            # 1. Calculate sum for j < i (L*x_new: using updated values)
            sum_L = np.dot(A[i, :i], x_new[:i])
            # 2. Calculate sum for j > i (U*x_old: using old values)
            sum_U = np.dot(A[i, i+1:], x_old[i+1:])

            # Update: x_i^(k+1) = (b_i - sum_L - sum_U) / A_ii
            x_new[i] = (b[i] - sum_L - sum_U) / A[i, i]

        # Relative Error Criterion (L_inf norm)
        error_norm = np.linalg.norm(x_new - x_old, np.inf)
        if error_norm < tol * np.linalg.norm(x_new, np.inf):
            # print(f"Gauss-Seidel converged in {k} iterations.")
            return x_new, k

        x_old = x_new

    # print(f"Gauss-Seidel failed to converge in {max_iter} iterations.")
    return x_old, max_iter
```

Stopping Criteria and Error Norms

The convergence condition is $\rho(\mathbf{T}) < 1$, which means $\|\mathbf{e}^{(k)}\| \to 0$. In practice, we halt the process when the error is sufficiently small.

Definition 1.1.12 (Stopping Criteria)**.** We use a relative error or residual criterion:

1. **Relative Error Criterion (Practical):** Stop when the difference between successive iterates is small:

$$\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| < \text{tol} \cdot \|\mathbf{x}^{(k+1)}\|$$

   where $\|\cdot\|$ is typically the ℓ_2 or ℓ_∞ norm.

2. **Residual Criterion (More Robust):** Stop when the residual vector $\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}$ is small:

$$\frac{\|\mathbf{r}^{(k)}\|}{\|\mathbf{b}\|} < \text{tol}$$

   This is often preferred as it directly relates to how well the system $\mathbf{A}\mathbf{x} = \mathbf{b}$ is satisfied.

The choice of the matrix norm (e.g., ℓ_∞ norm in the SDD proof) is usually dictated by analytical convenience, but the choice of the vector norm for the stopping criterion is a practical trade-off, with ℓ_2 and ℓ_∞ being most common.

1.2 Direct Methods: LU Factorization

While Iterative Methods leverage sparsity for efficiency in large-scale systems, they rely on favorable spectral properties (i.e., $\rho(\mathbf{T}) < 1$). For dense, small-to-moderately sized systems, or when the matrix $\mathbf{A}$ is ill-conditioned (and thus, $\rho(\mathbf{T}) \approx 1$), direct methods are computationally stable and preferred. These methods terminate in a finite number of operations, primarily relying on the Gaussian Elimination process, which is fundamentally an implicit matrix factorization.

Definition 1.2.1 (LU Factorization)**.** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be a square matrix. An **LU** factorization of $\mathbf{A}$ is a decomposition $\mathbf{A} = \mathbf{LU}$, where:

1. $\mathbf{L} \in \mathbb{R}^{n\times n}$ is a Unit Lower-Triangular Matrix: $L_{ij} = 0$ for $i < j$ and $L_{ii} = 1\ \forall i$.
2. $\mathbf{U} \in \mathbb{R}^{n\times n}$ is an Upper-Triangular Matrix: $U_{ij} = 0$ for $i > j$.

If a standard LU decomposition exists, the solution to $\mathbf{Ax} = \mathbf{b}$ is reduced to two simpler triangular solves:

1. Forward Substitution: Solve $\mathbf{Ly} = \mathbf{b}$ for the intermediate vector $\mathbf{y}$.
2. Backward Substitution: Solve $\mathbf{Ux} = \mathbf{y}$ for the solution vector $\mathbf{x}$.

Theorem 1.2.2 (Existence and Uniqueness of Standard LU Factorization)**.** *An $n \times n$ matrix $\mathbf{A}$ admits an $\mathbf{LU}$ factorization (with $\mathbf{L}$ unit lower-triangular) if and only if all of its leading principal minors are non-zero. That is,*

$$\det(\mathbf{A}_k) \neq 0 \quad \textit{for all } k = 1, \dots, n-1$$

where $\mathbf{A}_k$ is the $k \times k$ submatrix in the upper-left corner of $\mathbf{A}$. If such a factorization exists, it is unique.

Proof Sketch (Constructive). The proof is constructive and relies on the Gaussian elimination process without row interchanges. In the k-th step of elimination, we form the matrix $\mathbf{M}_k = \mathbf{I} - \mathbf{m}_k\mathbf{e}_k^T$, where $\mathbf{m}_k = [0, \dots, 0, m_{k+1,k}, \dots, m_{n,k}]^T$ are the multipliers that zero out entries below the pivot $A_{k,k}^{(k-1)}$. This step requires the pivot to be non-zero, $A_{k,k}^{(k-1)} \neq 0$. The resulting upper-triangular matrix is $\mathbf{U} = \mathbf{M}_{n-1}\cdots\mathbf{M}_1\mathbf{A}$. Since each $\mathbf{M}_k$ is unit lower-triangular, their product $\mathbf{M} = \mathbf{M}_{n-1}\cdots\mathbf{M}_1$ is also unit lower-triangular. Thus, $\mathbf{A} = \mathbf{M}^{-1}\mathbf{U}$. The matrix $\mathbf{L} = \mathbf{M}^{-1}$ is easily computed as $\mathbf{L} = \mathbf{I} + \mathbf{m}_1\mathbf{e}_1^T + \cdots + \mathbf{m}_{n-1}\mathbf{e}_{n-1}^T$, where the entries below the diagonal are simply the multipliers used during elimination.

The condition $\det(\mathbf{A}_k) \neq 0$ ensures that all necessary pivots are non-zero, guaranteeing the process can continue up to $n-1$. The determinant of $\mathbf{A}_k$ is equal to the product of the first k pivots: $\det(\mathbf{A}_k) = A_{1,1}^{(0)}A_{2,2}^{(1)}\cdots A_{k,k}^{(k-1)}$. Thus, non-zero principal minors are equivalent to the existence of non-zero pivots. □

Computational Complexity of LU Decomposition

The LU decomposition is a process of $\mathcal{O}(n^3)$ complexity.

Proposition 1.2.3 (Computational Cost of LU)**.** *The total number of floating-point operations (FLOPs) required to compute the $\mathbf{L}$ and $\mathbf{U}$ factors for a dense $n \times n$ matrix is asymptotically:*

$$\textit{FLOPs}(\mathbf{LU}) \sim \frac{2}{3}n^3$$

The subsequent forward and backward substitutions required to solve $\mathbf{Ax} = \mathbf{b}$ *have a cost of* $2n^2$ *FLOPs.*

Remark 1.2.4. In data science, the high $\mathcal{O}(n^3)$ cost of the decomposition means that for a single system solve, the overhead dominates. However, if the same matrix $\mathbf{A}$ must be inverted or if multiple systems with the same $\mathbf{A}$ but different right-hand sides $\mathbf{b}$ must be solved, the $\mathcal{O}(n^2)$ cost of the triangular solves makes the $\mathbf{LU}$ factorization highly efficient. This re-use of the factorisation is a common pattern in iterative refinement, preconditioners, and time-stepping numerical methods.

1.2.1 Canonical Factorization Variants: Doolittle, Crout, and Cholesky

The standard LU factorization is not unique unless a constraint is imposed on the diagonal elements of $\mathbf{L}$ or $\mathbf{U}$. The three canonical forms impose different normalization constraints, which alter the computational structure and memory access patterns of the resulting algorithms.

Doolittle Factorization (Unit Lower)

The Doolittle factorization is the standard form derived directly from Gaussian Elimination, where the lower-triangular factor $\mathbf{L}$ is constrained to be Unit Lower-Triangular ($L_{ii} = 1$).

$$\mathbf{A} = \mathbf{LU} \quad \text{with } L_{ii} = 1$$

The entries are calculated recursively from the matrix equation $\mathbf{A}_{ij} = \sum_{k=1}^{\min(i,j)} L_{ik}U_{kj}$. The system of equations is:

$$U_{ij} = A_{ij} - \sum_{k=1}^{i-1} L_{ik}U_{kj} \quad (\text{For } i \leq j \text{ - Row } i \text{ of } \mathbf{U}) \tag{1.1}$$

$$L_{ij} = \frac{1}{U_{jj}}\left(A_{ij} - \sum_{k=1}^{j-1} L_{ik}U_{kj}\right) \quad (\text{For } i > j \text{ - Column } j \text{ of } \mathbf{L}) \tag{1.2}$$

The Doolittle algorithm, in its most common form, is a **right-looking factorization**. At each step k, it computes the k-th row of $\mathbf{U}$ and the k-th column of $\mathbf{L}$. It then immediately uses these to perform a rank-1 update on the remaining submatrix. This structure mirrors the standard Gaussian elimination process.

Crout Factorization (Unit Upper)

The Crout factorization imposes the constraint that the upper-triangular factor $\mathbf{U}$ is Unit Upper-Triangular ($U_{ii} = 1$).

$$\mathbf{A} = \mathbf{LU} \quad \text{with } U_{ii} = 1$$

The resulting system of equations is:

$$L_{ij} = A_{ij} - \sum_{k=1}^{j-1} L_{ik}U_{kj} \quad (\text{For } i \geq j \text{ - Column } j \text{ of } \mathbf{L}) \tag{1.3}$$

$$U_{ij} = \frac{1}{L_{ii}}\left(A_{ij} - \sum_{k=1}^{i-1} L_{ik}U_{kj}\right) \quad (\text{For } i < j \text{ - Row } i \text{ of } \mathbf{U}) \tag{1.4}$$

The Crout Algorithm is a classic **left-looking factorization**. To compute the j-th column of $\mathbf{L}$ and the j-th row of $\mathbf{U}$, it accesses all previously computed columns from 1 to $j-1$ ("looking left"). The calculation is based on dot products with these finalized columns. This method is often preferred in high-performance computing due to its excellent memory access patterns, which utilize cache and vectorization more effectively.

Remark 1.2.5 (Algorithmic Difference). The difference between Crout and Doolittle is simply an algebraic re-organization of the same $\mathcal{O}(n^3)$ calculation. Doolittle computes the multipliers L_{ij} and then uses them to find the entries of $\mathbf{U}$, matching the structure of classical Gaussian elimination. Crout flips the order, computing the full column of $\mathbf{L}$ first.

Cholesky Factorization: The Symmetric Case

The most important and stable variant for many data science and optimisation problems is the Cholesky Factorization, which applies to Symmetric Positive Definite (SPD) matrices. The symmetry constraint allows for a significantly more efficient and stable factorization.

Definition 1.2.6 (Cholesky Factorization)**.** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be an **SPD** matrix. The Cholesky factorization of $\mathbf{A}$ is a decomposition

$$\mathbf{A} = \mathbf{L}\mathbf{L}^T$$

where $\mathbf{L}$ is a Lower-Triangular Matrix with strictly positive diagonal entries ($L_{ii} > 0$).

Theorem 1.2.7 (Existence and Uniqueness of Cholesky Factorization)**.** *A square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ has a unique Cholesky factorization $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ if and only if $\mathbf{A}$ is Symmetric Positive Definite (SPD).*

*Proof of Existence (***SPD** $\implies$ *Cholesky).* The proof is an inductive construction (similar to the LU proof). The base case $n = 1$ is trivial: $A_{11} = L_{11}^2$, so $L_{11} = \sqrt{A_{11}}$, which is real and positive since $\mathbf{A}$ is SPD ($A_{11} > 0$).

For the inductive step, assume that every $k \times k$ principal submatrix of $\mathbf{A}$ is SPD and has a Cholesky factorization. We partition $\mathbf{A}$ as:

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{n-1} & \mathbf{v} \\ \mathbf{v}^T & a_{nn} \end{pmatrix}$$

where $\mathbf{A}_{n-1}$ is the $(n-1)\times(n-1)$ leading principal submatrix, $\mathbf{v} \in \mathbb{R}^{n-1}$, and $a_{nn} \in \mathbb{R}$. By the inductive hypothesis, $\mathbf{A}_{n-1}$ has a unique Cholesky factorization $\mathbf{A}_{n-1} = \mathbf{L}_{n-1}\mathbf{L}_{n-1}^T$. We seek a factor $\mathbf{L}$ of the form:

$$\mathbf{L} = \begin{pmatrix} \mathbf{L}_{n-1} & \mathbf{0} \\ \mathbf{u}^T & l_{nn} \end{pmatrix}$$

where $\mathbf{u} \in \mathbb{R}^{n-1}$ and $l_{nn} > 0$. The factorization $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ yields:

$$\begin{pmatrix} \mathbf{A}_{n-1} & \mathbf{v} \\ \mathbf{v}^T & a_{nn} \end{pmatrix} = \begin{pmatrix} \mathbf{L}_{n-1} & \mathbf{0} \\ \mathbf{u}^T & l_{nn} \end{pmatrix}\begin{pmatrix} \mathbf{L}_{n-1}^T & \mathbf{u} \\ \mathbf{0}^T & l_{nn} \end{pmatrix} = \begin{pmatrix} \mathbf{L}_{n-1}\mathbf{L}_{n-1}^T & \mathbf{L}_{n-1}\mathbf{u} \\ \mathbf{u}^T\mathbf{L}_{n-1}^T & \mathbf{u}^T\mathbf{u} + l_{nn}^2 \end{pmatrix}$$

Equating the blocks:

1. $\mathbf{A}_{n-1} = \mathbf{L}_{n-1}\mathbf{L}_{n-1}^T$ (Inductive hypothesis holds).

2. $\mathbf{v} = \mathbf{L}_{n-1}\mathbf{u}$. Since $\mathbf{L}_{n-1}$ is non-singular (it has a full rank, positive-diagonal triangular matrix), $\mathbf{u}$ is uniquely determined by the forward solve: $\mathbf{u} = \mathbf{L}_{n-1}^{-1}\mathbf{v}$.

3. $a_{nn} = \mathbf{u}^T\mathbf{u} + l_{nn}^2$. Thus, $l_{nn}^2 = a_{nn} - \mathbf{u}^T\mathbf{u}$.

For l_{nn} to be real and positive, we require $l_{nn}^2 = a_{nn} - \mathbf{u}^T\mathbf{u} > 0$. The quantity $\sigma = a_{nn} - \mathbf{u}^T\mathbf{u}$ is precisely the Schur complement of $\mathbf{A}$ with respect to $\mathbf{A}_{n-1}$. Since $\mathbf{A}$ is SPD, its Schur complement must also be positive, guaranteeing $l_{nn} = \sqrt{\sigma}$ is real and positive. Uniqueness follows from the fact that at each step, $\mathbf{u}$ and l_{nn} are uniquely defined. □

Cholesky Algorithm (Dot Product/Row-Oriented) The Cholesky algorithm simplifies the LU equations significantly. We only need to compute the entries of $\mathbf{L}$. The equations derived from $\mathbf{A}_{ij} = \sum_{k=1}^{j} L_{ik}L_{jk}$ (since $L_{ik}L_{jk} = L_{ik}(L^T)_{kj}$) are:

- Diagonal Elements ($i = j$):

$$A_{ii} = \sum_{k=1}^{i-1} L_{ik}^2 + L_{ii}^2 \implies L_{ii} = \sqrt{A_{ii} - \sum_{k=1}^{i-1} L_{ik}^2}$$

- Off-Diagonal Elements ($i > j$):

$$A_{ij} = \sum_{k=1}^{j-1} L_{ik}L_{jk} + L_{ij}L_{jj} \implies L_{ij} = \frac{1}{L_{jj}}\left(A_{ij} - \sum_{k=1}^{j-1} L_{ik}L_{jk}\right)$$

Proposition 1.2.8 (Computational Cost of Cholesky)**.** *Due to the symmetry and the fact that only* $\mathbf{L}$ *needs to be stored and computed, the Cholesky factorization is twice as fast as the general LU factorization. The total number of FLOPs is asymptotically:*

$$FLOPs(\mathbf{L}\mathbf{L}^T) \sim \frac{1}{3}n^3$$

This $\mathcal{O}(n^3/3)$ *complexity makes it the method of choice for any SPD system.*

1.2.2 Implementation and Numerical Stability: Pivoting and Condition Number

The Necessity of Pivoting: Factorization Failure

The fundamental flaw in a non-pivoted LU factorization is the potential for division by zero if a pivot U_{kk} (which is $\det(\mathbf{A}_k)/\det(\mathbf{A}_{k-1})$) is zero.

Definition 1.2.9 (Pivoting)**. Pivoting** is the strategic interchange of rows (and/or columns) of the matrix $\mathbf{A}$ to ensure that the factorization proceeds without breakdown and to enhance numerical stability.

- Partial Pivoting (Row Pivoting): At step k, the pivot row is swapped with a row $i \geq k$ such that $|A_{ik}|$ is maximized. This results in the **LUP** factorization (sometimes called **PLU**, personally I don't like this name due to the order implications, we do firts **PA** and after that, we look for the **LU** decomposition).

- Complete Pivoting (Row and Column Pivoting): At step k, the pivot is chosen from the entire un-eliminated submatrix to maximize $|A_{ij}|$, requiring both row and column swaps. This results in a $\mathbf{PAQ} = \mathbf{LU}$, which is called **LUPQ** decomposition (MATLAB names it LUCP).

Proposition 1.2.10 (Stability of LU)**.** *The standard, non-pivoted LU factorization is unstable in finite-precision arithmetic. The growth factor,* $\rho = \max_{i,j,k} |A_{ij}^{(k)}| / \max_{i,j} |A_{ij}^{(0)}|$*, can grow exponentially with* n*. Partial pivoting ensures that the multipliers in* $\mathbf{L}$ *are bounded by unity,* $|L_{ij}| \leq 1$*. Although* ρ *can still grow polynomially,* $\rho \leq 2^{n-1}$*, this bound is rarely reached in practice. For all practical purposes, Partial Pivoting is essential for robust LU-based solves in general-purpose computing.*

Remark 1.2.11 (Cholesky and Stability). The Cholesky factorization **does not require pivoting**. The fact that $\mathbf{A}$ is SPD inherently guarantees numerical stability. All entries of $\mathbf{L}$ are bounded, and no growth factor analysis is needed. This is a significant advantage of the Cholesky method.

The Role of Condition Number in Numerical Accuracy

In numerical analysis, a small residual $\|\mathbf{b} - \mathbf{A}\mathbf{x}_{comp}\|$ does not guarantee an accurate solution $\mathbf{x}_{comp}$. The error is magnified by the matrix's condition number.

Definition 1.2.12 (Condition Number)**.** For a non-singular matrix $\mathbf{A}$, the condition number with respect to a matrix norm $\|\cdot\|$ is defined as:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\|$$

If $\kappa(\mathbf{A})$ is large, the matrix is considered ill-conditioned.

Theorem 1.2.13 (Bound on Relative Error)**.** *Let* $\mathbf{x}$ *be the true solution to* $\mathbf{A}\mathbf{x} = \mathbf{b}$*, and* $\mathbf{x}_{comp}$ *be the computed solution. The relative error in the solution is bounded by the condition number multiplied by the relative residual:*

$$\frac{\|\mathbf{x} - \mathbf{x}_{comp}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A}) \frac{\|\mathbf{b} - \mathbf{A}\mathbf{x}_{comp}\|}{\|\mathbf{b}\|}$$

This theorem is central to understanding the limits of all direct methods: a large condition number implies that the relative accuracy of the solution degrades linearly with the condition number, regardless of how small the residual is.

1.2.3 Application in Data Science: Inversion, Least Squares, and Statistics

LU factorizations are the foundational workhorse for a wide array of data science and statistical computing tasks.

Matrix Inversion

While inverting a matrix $\mathbf{A}$ explicitly ($\mathbf{A}^{-1}$) is generally avoided due to stability and computational cost (requiring $\mathcal{O}(n^3)$ FLOPs), the $\mathbf{LU}$ factorization is the numerical mechanism by which inversion is achieved.

$$\mathbf{A}^{-1} = (\mathbf{LU})^{-1} = \mathbf{U}^{-1}\mathbf{L}^{-1}$$

The columns of $\mathbf{A}^{-1}$, denoted $\mathbf{z}_j$, are found by solving n systems $\mathbf{A}\mathbf{z}_j = \mathbf{e}_j$ (where $\mathbf{e}_j$ is the j-th standard basis vector), each requiring one forward and one backward solve ($\mathcal{O}(n^2)$ FLOPs). Total cost is $\mathcal{O}(n^3)$.

Solving the Normal Equations for Least Squares

The canonical linear regression problem in data science seeks to find the vector $\mathbf{x}$ that minimizes the ℓ_2-norm of the residual:

$$\min_{\mathbf{x}} \|\mathbf{y} - \mathbf{X}\mathbf{x}\|_2$$

where $\mathbf{X} \in \mathbb{R}^{m \times n}$ is the design matrix ($m \geq n$), and $\mathbf{y} \in \mathbb{R}^m$. The solution $\mathbf{x}$ satisfies the Normal Equations:

$$(\mathbf{X}^T\mathbf{X})\mathbf{x} = \mathbf{X}^T\mathbf{y}$$

If $\mathbf{X}$ is full-rank, the matrix $\mathbf{A} = \mathbf{X}^T\mathbf{X} \in \mathbb{R}^{n \times n}$ is guaranteed to be Symmetric Positive Definite (SPD).

1. Factorization: Compute the Cholesky factorization $\mathbf{A} = \mathbf{L}\mathbf{L}^T$. ($\mathcal{O}(n^3/3)$)
2. Solve: Solve $\mathbf{L}\mathbf{y} = \mathbf{X}^T\mathbf{y}$ (Forward) and $\mathbf{L}^T\mathbf{x} = \mathbf{y}$ (Backward). ($\mathcal{O}(n^2)$)

The use of Cholesky factorization is the default robust method for solving the normal equations in statistical software. However, it is important to note that the condition number of $\mathbf{X}^T\mathbf{X}$ is $\kappa(\mathbf{X}^T\mathbf{X}) = \kappa(\mathbf{X})^2$, meaning this approach is highly sensitive to ill-conditioning in the design matrix $\mathbf{X}$. Alternative methods, such as the QR factorization (to be covered in the next section), are generally preferred for ill-conditioned problems.

Determinant Calculation

The family of LU factorizations provides the most numerically efficient means to calculate the determinant of a matrix $\mathbf{A}$. The method leverages two key properties: the determinant of a product of matrices is the product of their determinants, and the determinant of a triangular matrix is the product of its diagonal entries.

$$\det(\mathbf{A}) = \det(\mathbf{L}\mathbf{U}) = \det(\mathbf{L})\det(\mathbf{U})$$

The final calculation depends on the specific constraints of the factorization used:

- **Doolittle Form ($L_{ii} = 1$):** Since $\mathbf{L}$ is a unit lower-triangular matrix, its determinant is trivially $\det(\mathbf{L}) = \prod_{i=1}^n L_{ii} = 1$. The calculation thus simplifies to the product of the diagonal entries of $\mathbf{U}$ (the pivots):

$$\det(\mathbf{A}) = \det(\mathbf{U}) = \prod_{i=1}^n U_{ii}$$

- **Crout Form ($U_{ii} = 1$):** Conversely, in the Crout form, $\mathbf{U}$ is unit upper-triangular, making its determinant $\det(\mathbf{U}) = \prod_{i=1}^n U_{ii} = 1$. The determinant of $\mathbf{A}$ is therefore the product of the diagonal entries of $\mathbf{L}$:

$$\det(\mathbf{A}) = \det(\mathbf{L}) = \prod_{i=1}^n L_{ii}$$

- **Cholesky Form ($\mathbf{A} = \mathbf{L}\mathbf{L}^T$):** For Symmetric Positive Definite matrices, we use the property that $\det(\mathbf{M}^T) = \det(\mathbf{M})$. The calculation becomes:

$$\det(\mathbf{A}) = \det(\mathbf{L}\mathbf{L}^T) = \det(\mathbf{L})\det(\mathbf{L}^T) = (\det(\mathbf{L}))^2$$

  Since $\mathbf{L}$ is lower-triangular with positive diagonal entries, the determinant is the square of the product of its diagonal elements:

$$\det(\mathbf{A}) = \left(\prod_{i=1}^{n} L_{ii}\right)^2$$

  This elegantly confirms that the determinant of an SPD matrix is always positive.

If pivoting is used ($\mathbf{PA} = \mathbf{LU}$), we must account for the determinant of the permutation matrix $\mathbf{P}$. Since $\det(\mathbf{P}) = (-1)^s = (-1)^{-s} \quad \forall s \in \mathbb{N}$, where s is the number of row interchanges, the formula becomes:

$$\det(\mathbf{P})\det(\mathbf{A}) = \det(\mathbf{L})\det(\mathbf{U}) \implies \det(\mathbf{A}) = (-1)^s \det(\mathbf{L})\det(\mathbf{U})$$

For the standard pivoted LU factorization (Doolittle form), this is $\det(\mathbf{A}) = (-1)^s \prod_{i=1}^{n} U_{ii}$. The overall calculation remains dominated by the $\mathcal{O}(n^3)$ cost of the initial factorization and is the standard numerical method for computing determinants.

1.2.4 Numerical Implementation in Python

To solidify the theoretical foundations, we present pedagogical implementations of the Doolittle, Crout, and Cholesky factorizations. These functions are written "from scratch" using the `numpy` library to directly mirror the underlying mathematical formulas. We also include the requisite forward and backward substitution solvers that utilize these factorizations to solve linear systems.

Core Solver: Substitution Algorithms

The efficiency of the LU factorization paradigm hinges on the speed of solving the resulting triangular systems. The following functions implement forward substitution for lower-triangular systems ($\mathbf{Ly} = \mathbf{b}$) and backward substitution for upper-triangular systems ($\mathbf{Ux} = \mathbf{y}$).

```python
import numpy as np

def forward_substitution(L, b):
    """
    Solves the system Ly = b for y, where L is a lower-triangular matrix.
    Assumes L_ii is not zero. For Doolittle, L_ii = 1.
    """
    n = L.shape[0]
    y = np.zeros(n, dtype=L.dtype)
    for i in range(n):
        # The @ operator is dot product for 1D arrays
        y[i] = (b[i] - L[i, :i] @ y[:i]) / L[i, i]
```

```
    return y

def backward_substitution(U, y):
    """
    Solves the system Ux = y for x, where U is an upper-triangular matrix.
    Assumes U_ii is not zero.
    """
    n = U.shape[0]
    x = np.zeros(n, dtype=U.dtype)
    for i in range(n - 1, -1, -1):
        x[i] = (y[i] - U[i, i+1:] @ x[i+1:]) / U[i, i]
    return x
```

Factorization Algorithms

The following functions implement the three canonical factorization variants discussed. Note that these are non-pivoted implementations and will fail if a zero pivot is encountered (for LU/Crout) or if the matrix is not SPD (for Cholesky).

Doolittle Factorization ($L_{ii} = 1$) This implementation follows the right-looking (Gaussian elimination) structure.

```
def doolittle_lu(A):
    """
    Performs LU factorization of A using Doolittle's method (L_ii = 1).
    Returns L, U. This is a non-pivoted implementation.
    """
    n = A.shape[0]
    L = np.eye(n, dtype=A.dtype)
    U = np.zeros((n, n), dtype=A.dtype)

    for k in range(n):
        # Calculate k-th row of U
        # U[k, k:] = A[k, k:] - sum(L[k, p]*U[p, k:]) for p=0..k-1
        # This is a dot product of the k-th row of L with columns of U
        U[k, k:] = A[k, k:] - L[k, :k] @ U[:k, k:]

        # Calculate k-th column of L
        if np.abs(U[k, k]) < 1e-12:
            raise np.linalg.LinAlgError("Zero pivot encountered.")

        # L[k+1:, k] = (A[k+1:, k] - sum(L[k+1:, p]*U[p, k])) / U[k, k]
        # This is a dot product of submatrix of L with k-th column of U
        L[k+1:, k] = (A[k+1:, k] - L[k+1:, :k] @ U[:k, k]) / U[k, k]

    return L, U
```

Crout Factorization ($U_{ii} = 1$) This implementation follows the left-looking (dot-product) structure.

```python
def crout_lu(A):
    """
    Performs LU factorization of A using Crout's method (U_ii = 1).
    Returns L, U. This is a non-pivoted implementation.
    """
    n = A.shape[0]
    L = np.zeros((n, n), dtype=A.dtype)
    U = np.eye(n, dtype=A.dtype)

    for j in range(n):
        # Calculate j-th column of L
        # L[j:, j] = A[j:, j] - sum(L[j:, k]*U[k, j]) for k=0..j-1
        L[j:, j] = A[j:, j] - L[j:, :j] @ U[:j, j]

        # Calculate j-th row of U
        if np.abs(L[j, j]) < 1e-12:
            raise np.linalg.LinAlgError("Zero pivot encountered.")

        # U[j, j+1:] = (A[j, j+1:] - sum(L[j, k]*U[k, j+1:])) / L[j, j]
        U[j, j+1:] = (A[j, j+1:] - L[j, :j] @ U[:j, j+1:]) / L[j, j]

    return L, U
```

Cholesky Factorization ($\mathbf{A} = \mathbf{L}\mathbf{L}^T$) This algorithm computes the lower-triangular factor $\mathbf{L}$ for an SPD matrix. It includes a check for positive definiteness by ensuring the arguments to the square root are positive.

```python
def cholesky(A):
    """
    Performs Cholesky factorization of A (A = LL^T).
    Returns the lower-triangular matrix L.
    A must be a symmetric positive-definite matrix.
    """
    n = A.shape[0]
    L = np.zeros((n, n), dtype=A.dtype)

    for i in range(n):
        for j in range(i + 1):
            # Sum of squares of elements in the same row
            s = L[i, :j] @ L[j, :j]

            if i == j: # Diagonal elements
                diag_val = A[i, i] - s
                if diag_val < 0:
                    raise np.linalg.LinAlgError("Matrix is not positive definite.")
                L[i, j] = np.sqrt(diag_val)
            else: # Off-diagonal elements
                if L[j, j] == 0:
```

```
                    raise np.linalg.LinAlgError("Matrix is not positive definite.")
                L[i, j] = (A[i, j] - s) / L[j, j]

    return L
```

Example Usage: Solving a System

Finally, we demonstrate how to combine these functions to solve a linear system $\mathbf{Ax} = \mathbf{b}$.

```
def solve_with_lu(factorization_func, A, b):
    """
    Solves Ax = b using a specified LU factorization function.
    """
    print(f"--- Solving with {factorization_func.__name__} ---")
    try:
        L, U = factorization_func(A)
        print("L:\n", np.round(L, 4))
        print("U:\n", np.round(U, 4))

        # Step 1: Solve Ly = b
        y = forward_substitution(L, b)

        # Step 2: Solve Ux = y
        x = backward_substitution(U, y)
        return x
    except np.linalg.LinAlgError as e:
        print(e)
        return None

# Example Matrix (Doolittle and Crout)
A = np.array([[2., 1., 1.], [4., 3., 3.], [8., 7., 9.]])
b = np.array([6., 14., 36.])

x_doolittle = solve_with_lu(doolittle_lu, A, b)
x_crout = solve_with_lu(crout_lu, A, b)

print("\nSolution (Doolittle):", np.round(x_doolittle, 4))
print("Solution (Crout):", np.round(x_crout, 4))
print("Verification (numpy.linalg.solve):", np.round(np.linalg.solve(A, b), 4))


# Example SPD Matrix (Cholesky)
A_spd = np.array([[4., 2., -2.], [2., 5., 5.], [-2., 5., 14.]])
b_spd = np.array([6., 27., 45.])

print("\n--- Solving with Cholesky ---")
try:
    L_cholesky = cholesky(A_spd)
    print("L (Cholesky):\n", np.round(L_cholesky, 4))
```

```
    # For A=LL^T, we solve Ly=b and then L^T x = y
    y = forward_substitution(L_cholesky, b_spd)
    x_cholesky = backward_substitution(L_cholesky.T, y) # Note the transpose!

    print("\nSolution (Cholesky):", np.round(x_cholesky, 4))
    print("Verification (numpy.linalg.solve):", np.round(np.linalg.solve(A_spd, b_spd
except np.linalg.LinAlgError as e:
    print(e)
```

1.2.5 ⋆ Further Variants and Generalizations of LU Factorization

Beyond the canonical Doolittle, Crout, and Cholesky forms, several other factorizations generalize or adapt the core LU concept. These variants are crucial in both theoretical and applied contexts, from ensuring universal applicability to optimizing for specific matrix structures. We state their foundational theorems without proof, highlighting the mathematical principles upon which they are built.

LDU Factorization: The Symmetric Form

This variant explicitly isolates the pivots (the diagonal of $\mathbf{U}$) into a separate diagonal matrix $\mathbf{D}$, leaving two unit triangular factors.

- **Constraint:** Both $\mathbf{L}$ and a new upper-triangular matrix $\mathbf{U}'$ are unit triangular, and $\mathbf{D}$ is a diagonal matrix.

$$\mathbf{A} = \mathbf{L}\mathbf{D}\mathbf{U}'$$

Theorem 1.2.14 (Existence and Uniqueness of LDU Factorization)**.** *If a square matrix* $\mathbf{A}$ *admits an LU factorization and is non-singular, then it has a unique factorization* $\mathbf{A} = \mathbf{L}\mathbf{D}\mathbf{U}'$*, where* $\mathbf{L}$ *and* $\mathbf{U}'$ *are unit triangular and* $\mathbf{D}$ *is diagonal.*

Remark 1.2.15 (Theoretical Insight)*.* This factorization is a simple algebraic rearrangement of the standard LU form ($\mathbf{U} = \mathbf{D}\mathbf{U}'$). Its primary theoretical importance is revealing the symmetric structure when $\mathbf{A}$ itself is symmetric. If $\mathbf{A} = \mathbf{A}^T$, then by the uniqueness of the LDU factorization, we must have $\mathbf{U}' = \mathbf{L}^T$, leading directly to the $\mathbf{L}\mathbf{D}\mathbf{L}^T$ form.

$\mathbf{LDL^T}$ Factorization: The Square-Root-Free Cholesky

This is a direct and computationally significant consequence of the LDU factorization for symmetric matrices. It retains the efficiency of Cholesky but avoids computing square roots.

- **Constraint:** The matrix $\mathbf{A}$ must be symmetric, $\mathbf{L}$ is unit lower-triangular, and $\mathbf{D}$ is diagonal (but not necessarily with positive entries).

$$\mathbf{A} = \mathbf{L}\mathbf{D}\mathbf{L}^T$$

Theorem 1.2.16 (Existence of $\mathrm{LDL^T}$ Factorization)**.** *A non-singular symmetric matrix* $\mathbf{A}$ *has a unique* $\mathbf{L}\mathbf{D}\mathbf{L}^T$ *factorization if and only if all its leading principal minors are non-zero.*

Remark 1.2.17 (Theoretical Insight). This factorization is more general than the standard Cholesky form. Since $\mathbf{D}$ is not required to be positive, the $\mathbf{LDL}^T$ decomposition exists for a broader class of matrices, including symmetric **indefinite** matrices. This is critically important in optimization algorithms (like Newton's method), where the Hessian matrix is symmetric but may not be positive definite. The proof is inherited directly from the LDU case. The signs of the diagonal entries in $\mathbf{D}$ (the pivots) constitute the **inertia** of the matrix, a concept from spectral theory.

Block LU Factorization

This generalization applies the factorization idea to matrices partitioned into sub-matrices or "blocks".

- **Constraint:** The matrix $\mathbf{A}$ is partitioned as $\mathbf{A} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix}$, where the leading principal block $\mathbf{A}_{11}$ is square and non-singular.

Theorem 1.2.18 (Existence of Block LU Factorization)**.** *If $\mathbf{A}_{11}$ is non-singular, then $\mathbf{A}$ can be factored as:*

$$\mathbf{A} = \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{A}_{21}\mathbf{A}_{11}^{-1} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{0} & \mathbf{A}_{22} - \mathbf{A}_{21}\mathbf{A}_{11}^{-1}\mathbf{A}_{12} \end{pmatrix}$$

*This is a block LU decomposition where the lower-right entry of the upper-triangular block is the **Schur Complement** of $\mathbf{A}_{11}$ in $\mathbf{A}$.*

Remark 1.2.19 (Theoretical Insight). This factorization is fundamental to many advanced numerical algorithms, including domain decomposition methods for solving PDEs and matrix inversion formulas. The proof is a direct algebraic verification via block matrix multiplication. The properties of the **Schur complement**, $S = \mathbf{A}_{22} - \mathbf{A}_{21}\mathbf{A}_{11}^{-1}\mathbf{A}_{12}$, are a deep field of study. For example, if $\mathbf{A}$ is Symmetric Positive Definite, then so is its Schur complement S. This property allows for recursive proofs and algorithms based on partitioning, such as those used in computational statistics for analyzing conditional probability distributions of multivariate Gaussian variables.

Factorization of Band Matrices

This is a curious and practically paramount specialization. Instead of a new form, it postulates that the *structure* of $\mathbf{A}$ is inherited by its factors $\mathbf{L}$ and $\mathbf{U}$.

- **Constraint:** The matrix $\mathbf{A}$ is a **band matrix**, meaning $A_{ij} = 0$ if $|i - j| > w$ for some bandwidth w.

Theorem 1.2.20 (Band Structure Preservation)**.** *If a band matrix $\mathbf{A}$ with lower bandwidth w_L and upper bandwidth w_U admits an LU factorization (without pivoting), then the factor $\mathbf{L}$ has lower bandwidth w_L and the factor $\mathbf{U}$ has upper bandwidth w_U.*

Remark 1.2.21 (Theoretical Insight). This theorem is profound because it states that the factorization process introduces no new non-zero entries outside the original band (a property known as zero "fill-in" outside the band). This preservation of sparsity reduces the computational cost dramatically from $\mathcal{O}(n^3)$ to $\mathcal{O}(n \cdot w_L \cdot w_U)$. The most celebrated

case is for tridiagonal matrices ($w_L = w_U = 1$), where the cost becomes linear, $\mathcal{O}(n)$. The resulting algorithm is known as the **Thomas Algorithm** or TDMA (Tridiagonal Matrix Algorithm), and it is the cornerstone for solving linear systems that arise from finite difference discretizations of Partial Differential Equations, such as the heat equation.

Rank-Revealing LU Factorization

This variant is a more ambitious postulation. Its objective is not just numerical stability, but to obtain structural information about the matrix, specifically its **numerical rank**.

- **Constraint:** Complete pivoting (both rows and columns) is used to maximize the magnitude of the pivot at each step of the process. The resulting form is $\mathbf{PAQ} = \mathbf{LU}$.

Theorem 1.2.22 (Rank-Revealing LU Structure)**.** *For any matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ of rank r, there exist permutation matrices $\mathbf{P}$ and $\mathbf{Q}$ such that in the factorization $\mathbf{PAQ} = \mathbf{LU}$, the matrix $\mathbf{U}$ has a structure that reveals the rank:*

$$\mathbf{U} = \begin{pmatrix} \mathbf{U}_{11} & \mathbf{U}_{12} \\ \mathbf{0} & \mathbf{U}_{22} \end{pmatrix}$$

where $\mathbf{U}_{11}$ is an $r \times r$ non-singular upper-triangular matrix, and the entries of $\mathbf{U}_{22}$ are numerically zero (or of very small magnitude).

Remark 1.2.23 (Theoretical Insight). The complete pivoting strategy acts as a greedy algorithm that attempts to concentrate the "energy" or the linearly independent information of the matrix into the upper-left block $\mathbf{U}_{11}$. The pivots (the diagonal of $\mathbf{U}$) tend to appear in decreasing order of magnitude, and an abrupt drop indicates the numerical rank r. While theoretically powerful, this factorization is computationally more expensive than partial pivoting. It is an important tool for diagnosing collinearity in statistical models and serves as a conceptual precursor to the Singular Value Decomposition (SVD), which is the most robust numerical method for determining rank. Its theoretical analysis relies on perturbation theory and the properties of element growth under Gaussian elimination with complete pivoting.

1.3 The QR Factorization: The Stable Path to Least Squares

While the Cholesky factorization of the normal equations, $\mathbf{A}^T\mathbf{A}\mathbf{x} = \mathbf{A}^T\mathbf{b}$, provides an elegant and efficient method for solving full-rank least-squares problems, it harbors a significant numerical pathology. The very formation of the Gram matrix $\mathbf{A}^T\mathbf{A}$ can dramatically worsen the conditioning of the problem.

Theorem 1.3.1 (Conditioning of Normal Equations)**.** *Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m \geq n$ be a full-rank matrix. The condition number of $\mathbf{A}^T\mathbf{A}$ is the square of the condition number of $\mathbf{A}$:*

$$\kappa_2(\mathbf{A}^T\mathbf{A}) = (\kappa_2(\mathbf{A}))^2$$

Remark 1.3.2. This squaring of the condition number is numerically catastrophic. If $\kappa_2(\mathbf{A}) = 10^4$ (a moderately ill-conditioned matrix), then $\kappa_2(\mathbf{A}^T\mathbf{A}) = 10^8$. This inflation can transform a solvable problem into one where floating-point arithmetic yields a solution with no correct digits. This motivates the search for a method that avoids forming $\mathbf{A}^T\mathbf{A}$ entirely. The QR factorization is the definitive, stable answer.

The core idea is to factor $\mathbf{A}$ into a product of a matrix with orthonormal columns and an upper-triangular matrix. Such a decomposition leverages the supreme numerical stability of orthogonal transformations.

Definition 1.3.3 (Reduced QR Factorization)**.** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m \geq n$ and full column rank. The **reduced QR factorization** (or thin QR) is a decomposition

$$\mathbf{A} = \mathbf{Q}_1\mathbf{R}_1$$

where:

- $\mathbf{Q}_1 \in \mathbb{R}^{m\times n}$ is a matrix with orthonormal columns. This implies $\mathbf{Q}_1^T\mathbf{Q}_1 = \mathbf{I}_n$.
- $\mathbf{R}_1 \in \mathbb{R}^{n\times n}$ is a non-singular, upper-triangular matrix.

The columns of $\mathbf{Q}_1$ form an orthonormal basis for the column space of $\mathbf{A}$, $\mathrm{Col}(\mathbf{A})$.

Remark 1.3.4 (The Theoretical Full QR Factorization). The reduced form can be extended to a **full factorization** $\mathbf{A} = \mathbf{QR}$, where $\mathbf{Q} \in \mathbb{R}^{m\times m}$ is a square orthogonal matrix and $\mathbf{R} \in \mathbb{R}^{m\times n}$ is upper-trapezoidal. Here, $\mathbf{Q}$ is partitioned as $\mathbf{Q} = [\mathbf{Q}_1|\mathbf{Q}_2]$, where the columns of $\mathbf{Q}_2$ form an orthonormal basis for the orthogonal complement of $\mathrm{Col}(\mathbf{A})$. For practical computation, especially when $m \gg n$, the full form is prohibitively expensive to compute and store, and the reduced form is almost always what is meant by "QR factorization".

1.3.1 Solving Least Squares with Reduced QR

The reduced QR factorization provides a direct, stable path to the least-squares solution. The objective is to find the vector $\mathbf{x}$ that minimizes the residual norm, which geometrically means that the residual vector $\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$ must be orthogonal to the column space of $\mathbf{A}$. Since the columns of $\mathbf{Q}_1$ form a basis for $\mathrm{Col}(\mathbf{A})$, this orthogonality condition is:

$$\mathbf{Q}_1^T(\mathbf{b} - \mathbf{A}\mathbf{x}) = \mathbf{0}$$

Rearranging and substituting $\mathbf{A} = \mathbf{Q}_1\mathbf{R}_1$:

$$\begin{aligned}\mathbf{Q}_1^T\mathbf{A}\mathbf{x} &= \mathbf{Q}_1^T\mathbf{b}\\ \mathbf{Q}_1^T(\mathbf{Q}_1\mathbf{R}_1)\mathbf{x} &= \mathbf{Q}_1^T\mathbf{b}\\ (\mathbf{Q}_1^T\mathbf{Q}_1)\mathbf{R}_1\mathbf{x} &= \mathbf{Q}_1^T\mathbf{b}\\ \mathbf{I}_n\mathbf{R}_1\mathbf{x} &= \mathbf{Q}_1^T\mathbf{b}\end{aligned}$$

This simplifies to the $n \times n$ upper-triangular system:

$$\mathbf{R}_1\mathbf{x} = \mathbf{Q}_1^T\mathbf{b}$$

This system can be solved efficiently and stably for $\mathbf{x}$ using backward substitution. The entire process avoids computing $\mathbf{A}^T\mathbf{A}$, thereby preserving the original problem's conditioning.

1.3.2 Methods of Construction

There are two main philosophies for constructing the reduced QR factorization: explicit orthogonalization via the Gram-Schmidt process, and implicit orthogonalization via a sequence of orthogonal transformations.

The Gram-Schmidt Process: A Geometric Construction

The Gram-Schmidt process is the classical method for converting a set of linearly independent vectors into an orthonormal set.

Classical Gram-Schmidt (CGS) The algorithm constructs the columns of $\mathbf{Q}_1$ and $\mathbf{R}_1$ sequentially. For each column $\mathbf{a}_k$ of $\mathbf{A}$, it subtracts the projections onto the previously found orthonormal vectors $\{\mathbf{q}_1, \ldots, \mathbf{q}_{k-1}\}$ and then normalizes the result. The coefficients of these projections form the entries of $\mathbf{R}_1$.

Remark 1.3.5 (Numerical Instability of CGS). In finite-precision arithmetic, CGS is notoriously unstable. When the columns of $\mathbf{A}$ are nearly collinear, the subtraction step suffers from catastrophic cancellation. This leads to a computed $\mathbf{Q}_1$ matrix whose columns are not orthogonal to machine precision, a fatal flaw known as "loss of orthogonality".

Modified Gram-Schmidt (MGS) MGS is an algebraic rearrangement of CGS that is numerically far more stable. Instead of repeatedly projecting the original vectors $\mathbf{a}_k$, MGS orthogonalizes the entire set of remaining vectors at each step. This seemingly minor change dramatically improves the preservation of orthogonality in the final $\mathbf{Q}_1$ matrix, making it a viable, though not optimal, algorithm.

Orthogonal Transformations: Householder and Givens

The modern, stable approach is to apply a sequence of simple orthogonal transformations to the matrix $\mathbf{A}$ to gradually introduce zeros below the diagonal, transforming it into $\mathbf{R}_1$.

$$\mathbf{Q}_n^T \cdots \mathbf{Q}_2^T\mathbf{Q}_1^T\mathbf{A} = \begin{pmatrix}\mathbf{R}_1\\ \mathbf{0}\end{pmatrix}$$

The product of these transformations forms the implicit orthogonal matrix $\mathbf{Q}$.

Householder Reflectors A Householder transformation is an orthogonal matrix that reflects a vector across a hyperplane. It is the workhorse for dense QR factorization.

Definition 1.3.6 (Householder Reflector)**.** A Householder reflector is a matrix of the form $\mathbf{P} = \mathbf{I} - 2\frac{\mathbf{v}\mathbf{v}^T}{\|\mathbf{v}\|_2^2}$ for some non-zero vector $\mathbf{v} \in \mathbb{R}^m$. It is symmetric and orthogonal ($\mathbf{P} = \mathbf{P}^T = \mathbf{P}^{-1}$).

The algorithm proceeds by applying n reflectors to $\mathbf{A}$. In step k, a reflector $\mathbf{P}_k$ is constructed to zero out all elements below the diagonal in column k. This transformation is then applied to the entire remaining submatrix.

Remark 1.3.7 (Implicit Q). Crucially, the full $m \times m$ matrix $\mathbf{Q}$ is never formed. To solve the least-squares system, we only need to compute the vector $\mathbf{Q}_1^T\mathbf{b}$. This is achieved by sequentially applying the transposes of the same Householder transformations to the vector $\mathbf{b}$. This implicit application is an $\mathcal{O}(mn)$ process, which is far more efficient than forming and multiplying by an explicit $\mathbf{Q}_1$. The vectors defining the reflectors are efficiently stored in the lower-triangular part of the factored matrix $\mathbf{A}$.

Givens Rotations A Givens rotation is an orthogonal matrix that performs a plane rotation. It is a more "surgical" tool than a reflector, designed to zero out a single specific element. While less efficient for dense matrices (~ 1.5 times the cost of Householder), Givens rotations are indispensable for matrices with special structures, such as sparse or banded matrices, as they preserve sparsity much better than the dense Householder reflectors.

1.3.3 Numerical Implementation in Python

We now provide pedagogical Python implementations of the reduced QR factorization algorithms. The Gram-Schmidt methods are presented to illustrate the theory, followed by the numerically robust Householder QR, which reflects the standard approach used in high-performance libraries.

Gram-Schmidt Algorithms

These algorithms explicitly construct the factors $\mathbf{Q}_1$ and $\mathbf{R}_1$ of the reduced QR factorization.

```
import numpy as np

def classical_gram_schmidt(A):
    """
    Computes the reduced QR factorization of A using Classical Gram-Schmidt.
    WARNING: This algorithm is numerically unstable.
    """
    m, n = A.shape
    Q = np.zeros((m, n), dtype=A.dtype)
    R = np.zeros((n, n), dtype=A.dtype)

    for j in range(n):
        v = A[:, j].copy()
```

```
        for i in range(j):
            R[i, j] = Q[:, i].T @ A[:, j] # R_ij = q_i^T a_j
            v -= R[i, j] * Q[:, i]      # v = v - proj(v)

        R[j, j] = np.linalg.norm(v)
        if R[j, j] < 1e-12:
            raise np.linalg.LinAlgError("Matrix is rank-deficient.")
        Q[:, j] = v / R[j, j]

    return Q, R

def modified_gram_schmidt(A):
    """
    Computes the reduced QR factorization of A using Modified Gram-Schmidt.
    This version is more numerically stable than the classical version.
    """
    m, n = A.shape
    Q = np.zeros((m, n), dtype=A.dtype)
    R = np.zeros((n, n), dtype=A.dtype)
    V = A.copy()

    for j in range(n):
        R[j, j] = np.linalg.norm(V[:, j])
        if R[j, j] < 1e-12:
            raise np.linalg.LinAlgError("Matrix is rank-deficient.")
        Q[:, j] = V[:, j] / R[j, j]

        # Orthogonalize remaining vectors against the new q_j
        for k in range(j + 1, n):
            R[j, k] = Q[:, j].T @ V[:, k]
            V[:, k] -= R[j, k] * Q[:, j]

    return Q, R
```

Householder QR Algorithm

The standard library approach does not form $\mathbf{Q}_1$ explicitly. Instead, it computes $\mathbf{R}_1$ while storing the vectors that define the Householder reflectors in the lower part of the matrix. $\mathbf{Q}_1^T$ is then applied implicitly when needed.

```
def householder_qr(A):
    """
    Computes the implicit QR factorization of A using Householder reflectors.
    The upper triangle of the returned matrix is R1.
    The Householder vectors are stored compactly in the lower triangle.
    """
    m, n = A.shape
    A_factored = A.copy()
```

```
    for k in range(n):
        x = A_factored[k:m, k]
        # Use np.copysign for a robust sign function
        v = x.copy()
        v[0] += np.copysign(np.linalg.norm(x), x[0])
        v /= np.linalg.norm(v)

        # Apply the reflection P*A = A - 2v(v^T A) to the submatrix
        sub_matrix = A_factored[k:m, k:n]
        update = 2 * np.outer(v, v.T @ sub_matrix)
        A_factored[k:m, k:n] -= update

        # Store the essential part of the Householder vector
        A_factored[k+1:m, k] = v[1:]

    return A_factored

def solve_with_householder(A_factored, b):
    """
    Solves the least-squares problem using the implicit Householder QR.
    """
    m, n = A_factored.shape
    b_transformed = b.copy()

    # 1. Compute Q1^T * b by applying reflectors
    for k in range(n):
        v = np.zeros(m - k)
        v[0] = 1.0
        v[1:] = A_factored[k+1:m, k]

        sub_vector = b_transformed[k:m]
        update = 2 * v * (v.T @ sub_vector)
        b_transformed[k:m] -= update

    # 2. Extract R1 and the relevant part of Q1^T * b
    R1 = np.triu(A_factored[:n, :])
    c1 = b_transformed[:n]

    # 3. Solve the upper-triangular system R1*x = c1
    return np.linalg.solve(R1, c1)
```

Example Usage: Solving Least Squares

We now use these functions to solve a least-squares problem and verify the results.

```
# Let's solve a least-squares problem min||Ax-b||
A = np.array([[1., 1.], [2., 1.], [3., 1.], [4., 1.]])
b = np.array([1., 3., 3., 5.])
```

```
# --- Using Modified Gram-Schmidt (explicit Q1, R1) ---
print("--- Solving with Modified Gram-Schmidt ---")
Q1_mgs, R1_mgs = modified_gram_schmidt(A)
c1_mgs = Q1_mgs.T @ b
x_mgs = np.linalg.solve(R1_mgs, c1_mgs)
print("Solution (MGS):", np.round(x_mgs, 4))

# --- Using Householder Reflectors (implicit Q1) ---
print("\n--- Solving with Householder QR ---")
A_factored = householder_qr(A)
x_hh = solve_with_householder(A_factored, b)
print("Solution (Householder):", np.round(x_hh, 4))

# --- Verification with NumPy's built-in lstsq solver ---
x_numpy, _, _, _ = np.linalg.lstsq(A, b, rcond=None)
print("\nSolution (NumPy lstsq):", np.round(x_numpy, 4))
```